

1. Tempo disponibile 120 minuti.
2. Non è possibile consultare appunti, slide, libri, persone, siti web, ecc.
3. Scrivere in modo leggibile, su ogni foglio, nome, cognome e numero di matricola.
4. Le soluzioni agli esercizi che richiedono di progettare un algoritmo devono:
 - spiegare a parole l'algoritmo (se utile, anche con l'aiuto di esempi o disegni),
 - fornire e commentare lo pseudo-codice (indicando il significato delle variabili),
 - calcolare la complessità (con tutti i passaggi matematici necessari),
 - se l'esercizio ammette più soluzioni, a soluzioni computazionalmente più efficienti e/o concettualmente più semplici sono assegnati punteggi maggiori.
5. Nel caso di utilizzo di tecniche differenti da quelle descritte nel materiale didattico del corso, come ad esempio tecniche suggerite da intelligenze artificiali generative tipo ChatGPT, dovete giustificare la vostra scelta commentando le differenze ed i vantaggi (utilizzando, ove applicabile, l'analisi dei costi computazionali) rispetto all'uso delle tecniche descritte a lezione.

IMPORTANTE: Risolvere gli esercizi 1–2 e gli esercizi 3–4 su fogli separati. Infatti, al termine, dovete consegnare gli esercizi 1–2 separatamente dagli esercizi 3–4.

1. Calcolare la complessità $T(n)$ del seguente algoritmo MYSTERY1:

Algorithm 1: MYSTERY1(INT n) $\rightarrow$ INT

```

if  $n \leq 0$  then
  | return 0
else
  | return MYSTERY1( $n - 1$ ) + MYSTERY2( $2^n$ ) +  $n^3 + n^2 + n + 1$ 

function MYSTERY2(INT  $n$ )  $\rightarrow$  INT
if  $n \leq 0$  then
  | return 1
else
  | return MYSTERY2( $n/2$ ) +  $n^3 + n^2 + n + 1$ 

```

Soluzione. Analizziamo innanzitutto il costo di MYSTERY2, che esegue una chiamata ricorsiva su un input pari alla metà del valore originale. Le restanti operazioni eseguite da MYSTERY2 hanno costo costante. L'equazione di ricorrenza associata a MYSTERY2 è:

$$T'(n) = \begin{cases} 1 & n \leq 1 \\ T'(n/2) + 1 & n > 1 \end{cases}$$

Tale ricorrenza può essere risolta mediante il Master Theorem:

$$\alpha = \log_2 1 = 0 = \beta \Rightarrow T'(n) = \Theta(n^\alpha \log n) = \Theta(\log n)$$

La funzione MYSTERY1 esegue una chiamata ricorsiva su un input ridotto di una unità. Il costo delle restanti operazioni è dominato dalla chiamata alla funzione MYSTERY2 su input 2^n . Poiché MYSTERY2 ha costo logaritmico, una chiamata su input 2^n ha costo lineare:

$$T'(2^n) = \Theta(\log 2^n) = \Theta(n \log 2) = \Theta(n)$$

Pertanto, l'equazione di ricorrenza associata a MYSTERY1 è:

$$T(n) = \begin{cases} 1 & n \leq 0 \\ T(n-1) + n & n > 0 \end{cases}$$

Tale ricorrenza può essere risolta mediante il metodo dell'iterazione:

$$\begin{aligned} T(n) &= T(n-1) + n && \text{Passo 1} \\ &= T(n-2) + (n-1) + n && \text{Passo 2} \\ &= T(n-3) + (n-2) + (n-1) + n && \text{Passo 3} \\ &\dots \\ &= T(n-i) + \sum_{k=0}^{i-1} (n-k) && \text{Passo } i\text{-esimo} \end{aligned}$$

La ricorsione termina quando $n-i=0$, cioè quando $i=n$. Sostituendo nella formula precedente otteniamo:

$$T(n) = T(n-i) + \sum_{k=0}^{i-1} (n-k) = T(0) + \sum_{k=0}^{n-1} (n-k) = 1 + \sum_{k=1}^n k = 1 + \frac{n(n+1)}{2} = \Theta(n^2)$$

Pertanto, la complessità dell'algoritmo MYSTERY1 è $\Theta(n^2)$.

2. Sia A un array di interi i cui valori appartengono all'intervallo $[a, b]$. Si assuma che i valori di A siano *uniformemente distribuiti* nell'intervallo $[a, b]$ nel seguente senso: dato il punto medio dell'intervallo, $q = (a+b)/2$, circa metà degli elementi dell'array sono $\leq q$, mentre la restante metà è $> q$. La stessa proprietà vale ricorsivamente per gli elementi di A i cui valori appartengono agli intervalli $[a, q]$ e $[q+1, b]$, e così via. Ad esempio, l'array

[1, 2, 3, 8, 9, 10, 4, 5, 6, 7]

soddisfa questa proprietà, poiché $q = (1+10)/2 = 5.5$ ($a=1$ e $b=10$) ed esattamente metà dei suoi elementi sono ≤ 5.5 . Inoltre, la stessa proprietà vale ricorsivamente per i due sottointervalli di valori. Viceversa, l'array

[1, 100, 2, 93, 94, 95, 96, 97, 98, 99]

non soddisfa tale proprietà, poiché $q = (1+100)/2 = 50.5$ e soltanto 2 dei suoi valori sono ≤ 50.5 .

Si vuole progettare un algoritmo di ordinamento che sfrutti questa proprietà. In particolare:

- a) Discutere come modificare l'algoritmo *quicksort* in modo da sfruttare tale proprietà. Non è necessario scrivere lo pseudocodice: è sufficiente descrivere le modifiche da apportare.
- b) Analizzare e confrontare la complessità computazionale nel caso pessimo del *quicksort* originale e della modifica proposta quando vengono applicati a un array che soddisfa la proprietà di distribuzione uniforme descritta sopra.

Soluzione.

- a) La proprietà di distribuzione uniforme permette di implementare una funzione *partition* per *quicksort* che suddivide sempre l'array in due sottoarray bilanciati. La modifica principale consiste nel fatto che, mentre la funzione *partition* originale utilizza come pivot un elemento dell'array, nella versione modificata il valore del pivot potrebbe anche non comparire nell'array. In particolare:
 - Si assuma che l'array A abbia dimensione n e che $a = \min(A)$ e $b = \max(A)$.
 - Si scelga come pivot il valore $q = (a+b)/2$.
 - Si partizioni l'array in modo che tutti gli interi $\leq q$ vengano collocati nella parte sinistra e tutti gli interi $> q$ nella parte destra.

La stessa procedura può essere applicata ricorsivamente ai sottoarray generati. È necessario calcolare il minimo e il massimo di ciascun sottoarray; tali valori possono essere determinati in tempo lineare e, volendo, anche durante la fase di partizionamento.

- b) Poiché gli interi presenti nell'array sono distribuiti uniformemente, la modifica descritta al punto a) produce sempre partizioni bilanciate. L'equazione di ricorrenza che descrive il costo nel caso ottimo, medio e pessimo del *quicksort* modificato è quindi:

$$T(n) = \begin{cases} 1 & n \leq 1 \\ 2T(n/2) + n & n > 1 \end{cases}$$

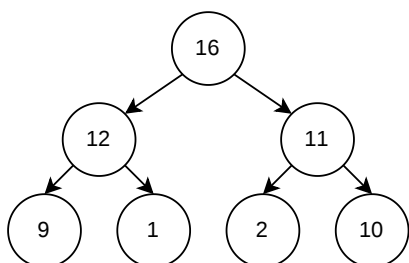
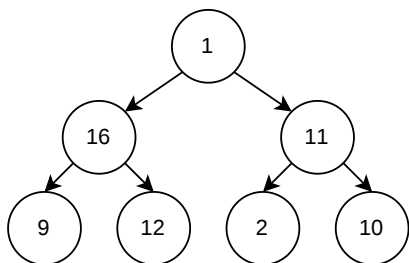
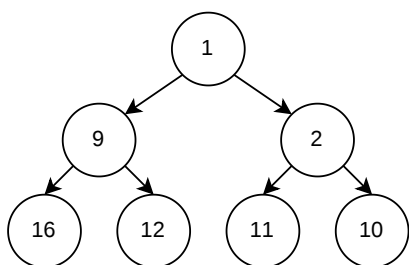
Tale ricorrenza può essere risolta mediante il Master Theorem:

$$\alpha = \log_2 2 = 1 = \beta \Rightarrow T(n) = \Theta(n^\alpha \log n) = \Theta(n \log n).$$

Il *quicksort* originale ha complessità pessima pari a $\Theta(n^2)$, mentre la versione modificata garantisce partizioni sempre bilanciate e raggiunge quindi una complessità pessima pari a $\Theta(n \log n)$, risultando asintoticamente ottimale tra gli algoritmi di ordinamento basati sul confronto.

3. Si consideri l'array $[1, 9, 2, 16, 12, 11, 10]$. Descrivere in che modo viene modificato l'array nel caso in cui venga eseguita su di esso la funzione *heapify* studiata a lezione per trasformarlo in un max-array-heap binario.

Soluzione. Per capire in che modo l'array viene modificato dalla funzione *heapify* si considera la rappresentazione dell'array come albero binario. Si ricorda che la funzione *heapify* esegue ricorsivamente dal basso l'operazione *fixHeap* su ogni sottoalbero, ad esclusione delle foglie.



Nella rappresentazione grafica si mostra prima l'albero binario iniziale. Successivamente si mostra come appare l'albero dopo aver eseguito *fixHeap* sui due sottoalberi della radice. Infine, si mostra l'albero dopo aver eseguito *fixHeap* sull'intero albero.

Al termine, l'array sarà quindi $[16, 12, 11, 9, 1, 2, 10]$.

4. Progettare un algoritmo che dato un grafo orientato $G = (V, E)$ e due vertici $v_1, v_2 \in V$ restituisce *true* se v_1 e v_2 appartengono alla medesima componente fortemente connessa, *false* altrimenti.

Soluzione. Per verificare se due vertici v_1 e v_2 appartengono alla medesima componente fortemente connessa è sufficiente verificare che siano vicendevolmente connessi, ovvero, che v_2 sia raggiungibile da v_1 e, vice versa, v_1 sia raggiungibile da v_2 . L'Algoritmo 2 effettua tale verifica utilizzando una funzione ausiliaria che verifica se a partire da un vertice v è possibile raggiungere un vertice u . Tale funzione utilizza una visita in ampiezza BFS che parte da v e termina restituendo *true* appena si incontra u , e restituisce *false* se si arriva al termine della visita senza incontrare u . Il costo computazionale di tale algoritmo è quindi dato del costo della visita BFS (che viene al più eseguita 2 volte, quindi una quantità costante di volte), ovvero $O(|V| + |E|)$ assumendo implementazione del grafo tramite liste di adiacenza.

Algorithm 2: VERIFICAFORTEMENTECONNESSI(GRAPH $G = (V, E)$, VERTEX v_1 , VERTEX v_2) $\rightarrow$ BOOL

return VERIFICARAGGIUNGIBILE(G, v_1, v_2) $\wedge$ VERIFICARAGGIUNGIBILE(G, v_2, v_1)

function VERIFICARAGGIUNGIBILE(GRAPH $G = (V, E)$, VERTEX v , VERTEX u) $\rightarrow$ BOOL

// Verifica se u è raggiungibile da v tramite BFS modificata

for $w \in V$ **do**

$w.visited \leftarrow false$

QUEUE $q \leftarrow$ **new** QUEUE()

$v.visited \leftarrow true$

$q.enqueue(v)$

while *not* $q.isEmpty()$ **do**

$t \leftarrow q.dequeue()$

for $w \in t.adjacents()$ **do**

if $w == v$ **then**

return *true*

if $w.visited == false$ **then**

$w.visited \leftarrow true$

$q.enqueue(w)$

return *false*
